Lab 3 — Viewing the Pi's `cv2.imshow` Window on Your Laptop (X11 over SSH)

This appendix explains how to set up **X11 forwarding** so that `pipeline_template.py --display` opens a native OpenCV window **on your laptop**, while the Pi 5 + Hailo-10H does all the inference.

```
Pi:  cv2.imshow → libX11 → sshd → (X11 over SSH) → your laptop's X server → window on your screen
```

No code changes are required — only laptop-side setup.

If you want to run onnx file on raspberry pi, you can install onnxruntime and opencv-python by running the following command:

```
pip install onnxruntime opencv-python numpy
```

0. Connect Raspberry Pi 5 to eduroam Wi-Fi

eduroam uses **WPA2-Enterprise (PEAP / MSCHAPv2)**, so a plain Wi-Fi password is not enough — you need your institutional username and password.

Run on the Pi (replace the placeholders):

```
sudo nmcli connection add \
  type wifi \
  ifname wlan0 \
  con-name eduroam \
  ssid "eduroam" \
  wifi-sec.key-mgmt wpa-eap \
  802-1x.eap peap \
  802-1x.phase2-auth mschapv2 \
  802-1x.identity "your_username@your.institution.edu" \
  802-1x.password "your_password" \
  802-1x.ca-cert "/etc/ssl/certs/ca-certificates.crt"
```

Bring the connection up:

```
sudo nmcli connection up eduroam
```

Verify:

```
nmcli device status          # wlan0 should show  connected  eduroam
ip addr show wlan0           # should display an assigned IP
```

To **remove** the profile later (e.g., to re-add with corrected credentials):

```
sudo nmcli connection delete eduroam
```

Troubleshooting

Symptom	Likely cause	Fix
`Error: Connection activation failed`	Wrong username / password	Double-check credentials; some institutions require `username@domain` format
Connects but no internet	CA certificate mismatch	Try removing the `ca_cert` line (less secure but useful for diagnosis) or obtain your institution's root CA
`wlan0` not found	Wi-Fi interface name differs	Run `ip link` to find the real name (e.g., `wlan1`) and update the command
Repeatedly asks for password	Phase 2 auth mismatch	Try `802-1x.phase2-auth mschapv2` vs `pap` depending on your institution

1. One-time Pi setup (all OSes)

Run **once** on the Raspberry Pi:

```
sudo apt update
sudo apt install -y xauth x11-apps    # xauth is usually already installed
```

Check that SSHD allows X11 forwarding (default on Raspberry Pi OS):

```
sudo grep -E '^X11Forwarding|^X11UseLocalhost' /etc/ssh/sshd_config
# Expected:
# "X11Forwarding yes"
# or "X11UseLocalhost yes"
```

If you had to change anything: `sudo systemctl restart ssh`.

Sanity test (run from your laptop **after** finishing the OS-specific section below):

```
ssh -Y <pi-user>@<pi-host>
echo $DISPLAY     # should print something like  localhost:10.0
xeyes             # a pair of googly eyes should appear on your laptop screen
```

If `xeyes` works, `cv2.imshow` will work.

2. macOS

Install XQuartz (the X server for macOS)

```
brew install --cask xquartz
```

If you do not use Homebrew, download the installer from https://www.xquartz.org/ and run it.

Log out of macOS and log back in (or reboot). XQuartz registers itself as the `$DISPLAY` handler only after a fresh login session.

Verify XQuartz

1. Open **XQuartz** from Applications → Utilities.
2. **XQuartz → Settings → Security**:
   - ✅ Authenticate connections
   - ☐ Allow connections from network clients *(leave OFF — SSH tunnels it)*
3. Open a fresh **Terminal** (not the XQuartz one) and run:

   ```
   echo $DISPLAY
   ```

   It should print something like `/private/tmp/com.apple.launchd.XXXX/org.xquartz:0`.
   If empty, log out and back in again.

Connect with X11 forwarding

```
ssh -Y <pi-user>@<pi-host>
```

- `-Y` = **trusted** X11 forwarding (faster, recommended for labs).
- `-X` works too but is slower due to extra security checks.

Optional — make it the default

Add this to `~/.ssh/config` on your Mac:

```
Host rasp
    HostName raspkit0.local
    User raspkit0
    ForwardX11 yes
    ForwardX11Trusted yes
```

Then just `ssh rasp` and you'll already have `$DISPLAY` set on the Pi.

Run the lab

```
ssh -Y rasp
xeyes                                          # sanity check
python3 pipeline_template.py --model yolo26n.hef --source picamera --display
```

An OpenCV window labeled **"YOLO @ Hailo"** opens on your Mac. Press `q` in that window to quit.

3. Windows

Windows has no built-in X server. Pick **one** of these:

Option A — VcXsrv (free, lightweight, recommended)

1. Download and install VcXsrv:
   https://sourceforge.net/projects/vcxsrv/
2. Launch **XLaunch** from the Start menu and configure it as follows:
   - Display settings: **Multiple windows**, Display number **0**
   - Client startup: **Start no client**
   - Extra settings:
     - ✅ Clipboard
     - ✅ Primary Selection
     - ✅ Native opengl
     - ✅ **Disable access control** *(important — otherwise SSH-forwarded clients are rejected)*
   - Click **Finish**. A small "X" icon appears in the system tray; leave VcXsrv running.
3. Allow VcXsrv through Windows Firewall when prompted (Private networks is enough).

Option B — MobaXterm (all-in-one SSH + X server, even simpler)

1. Download the **Home Edition** from https://mobaxterm.mobatek.net/.
2. Open MobaXterm. The built-in X server (`MobaX`) starts automatically — you'll see "X server: started" in the top status bar.
3. Click **Session** → **SSH**, set Remote host = `raspkit0.local` , user = `raspkit0` , tick **"X11-Forwarding"** under Advanced SSH settings.
4. Connect. `$DISPLAY` is already set; skip the rest of this section.

Option C — WSL2 + WSLg (Windows 11 only, no external X server)

If you're on Windows 11, you already have **WSLg**, which transparently forwards X11 and Wayland apps from WSL.

```
wsl --install -d Ubuntu        # if not installed yet
wsl
sudo apt update && sudo apt install -y openssh-client x11-apps
```

Then use the **Ubuntu instructions below** (section 4) from inside WSL.
No XLaunch / VcXsrv needed.

Connect with X11 forwarding (for Options A / B)

From **PowerShell**, **CMD**, or **Git Bash**:

```
set DISPLAY=127.0.0.1:0.0
ssh -Y <pi-user>@<pi-host>
```

In PowerShell you can also use `$env:DISPLAY = "127.0.0.1:0.0"` .

Inside the SSH session on the Pi:

```
echo $DISPLAY      # localhost:10.0
xeyes              # window should pop up on your Windows desktop

# Then press Ctrl + C to shutdown the xeyes
# Continue to run this command on the same ssh session:
export QT_QPA_PLATFORM=xcb # force Qt to use X11, not Wayland

# Then run all other commands related to LAB 3 also in the same ssh session.
# For example: python3 pipeline_template.py --model yolo26n.hef --source picamera --display ...
```

Optional — make it the default

In `%USERPROFILE%\.ssh\config` :

```
Host rasp
    HostName raspkit0.local
    User raspkit0
    ForwardX11 yes
    ForwardX11Trusted yes
```

And set `DISPLAY` once via **System Properties → Environment Variables**:
`DISPLAY = 127.0.0.1:0.0` (skip if using MobaXterm or WSLg).

Run the lab

```
ssh rasp
python3 pipeline_template.py --model yolo26n.hef --source picamera --display
```

4. Ubuntu (and most other Linux desktops)

X11 is usually already running, so this is the easiest case.

Check that you have an X server

```
echo $DISPLAY              # something like  :0   or  :1
xeyes                      # should open a window locally


# Then press Ctrl + C to shutdown the xeyes
# Continue to run this command on the same ssh session:
export QT_QPA_PLATFORM=xcb # force Qt to use X11, not Wayland

# Then run all other commands related to LAB 3 also in the same ssh session.
# For example: python3 pipeline_template.py --model yolo26n.hef --source picamera --display ...
```

If `xeyes` is missing: `sudo apt install -y x11-apps` .

Ubuntu 22.04+ / Fedora 36+ on Wayland

Modern Ubuntu often runs **Wayland** by default, but it still exposes an **XWayland** server so X11 forwarding still works — no action needed in most cases. If forwarding fails (`X11 forwarding request failed on channel`), log out and pick **"Ubuntu on Xorg"** from the gear icon on the login screen, *or* edit `/etc/gdm3/custom.conf` and set `WaylandEnable=false` .

Connect with X11 forwarding

```
ssh -Y <pi-user>@<pi-host>
echo $DISPLAY              # localhost:10.0 inside the SSH session
xeyes
```

Use `-X` instead of `-Y` if your distro complains about untrusted forwarding; `-Y` is faster but treats the remote as trusted.

Optional — make it the default

`~/.ssh/config` :

```
Host rasp
    HostName raspkit0.local
    User raspkit0
    ForwardX11 yes
    ForwardX11Trusted yes
```

Run the lab

```
ssh rasp
python3 pipeline_template.py --model yolo26n.hef --source picamera --display
```

5. Troubleshooting cheatsheet

Symptom	Cause	Fix
`qt.qpa.xcb: could not connect to display`	`$DISPLAY` is empty inside the SSH session	Reconnect with `ssh -Y` . On Windows, also start VcXsrv/MobaXterm first.
`echo $DISPLAY` empty even with `-Y`	X server not running on the laptop, or first login since installing XQuartz	macOS: log out / back in. Win: start VcXsrv. Linux: `xeyes` first to confirm local X.
`X11 forwarding request failed on channel 0`	sshd on Pi has `X11Forwarding no` , or `xauth` missing	`sudo apt install xauth` and set `X11Forwarding yes` in `/etc/ssh/sshd_config` , then `sudo systemctl restart ssh` .

Symptom	Cause	Fix
Window opens but is extremely laggy	Wi-Fi + uncompressed X11 traffic	Use wired Ethernet, lower `--capture-size`, or switch to the notebook/MJPEG view.
Worked once, broken after Pi reboot	Stale `~/.Xauthority`	On the Pi: `rm ~/.Xauthority`, reconnect with `ssh -Y`.
Qt picks Wayland instead of X	`WAYLAND_DISPLAY` set on the Pi	`export QT_QPA_PLATFORM=xcb` before running the script.
`Authorization required, but no authorization protocol specified`	VcXsrv launched without **Disable access control**	Re-run XLaunch and tick that box.
macOS only: window appears but mouse / keyboard ignored	XQuartz not focused	Click the window once; or in XQuartz → Settings → Input enable "Emulate three button mouse".

6. When not to use X11 forwarding

- More than one student needs to see the same demo simultaneously.
- The Wi-Fi link is slow and you want > 15 FPS.
- You don't want students installing extra software on their laptops.

In those cases use the in-notebook live view (`ipywidgets.Image`) or the MJPEG stream option instead. X11 is best as an **optional / advanced** path for students who already have a working X server.